

Machine Learning Week 3 by 21337470

Introduction

In this assignment, we were tasked with using Lasso and Ridge Regression algorithms. These are both forms of Linear Regression algorithms.

Lasso regression applies an L1 penalty, which encourages sparsity by driving some coefficients to zero, effectively selecting a simpler model by excluding irrelevant features. On the other hand, Ridge regression uses an L2 penalty, which shrinks the magnitude of the coefficients uniformly, allowing all features to contribute to the model, though at a smaller scale.

Cross Validation techniques are also used to get the best regularization parameter for the models.

Code Index : 1

The first section of the game is for the imports of dependencies used in this assignment.

First I imported pandas for manipulation of datasets (as pd), matplotlib for creating plots (as plt) and subsequently mplot3d extension of matplotlib, to create the 3D scatterplots used in this assignment.

Then I imported the NumPy which is used for array manipulation (as np).

After this, I imported the sklearn library and the models and metric functions from it as needed in this assignment which I will discuss later in this report.

Part (i). a

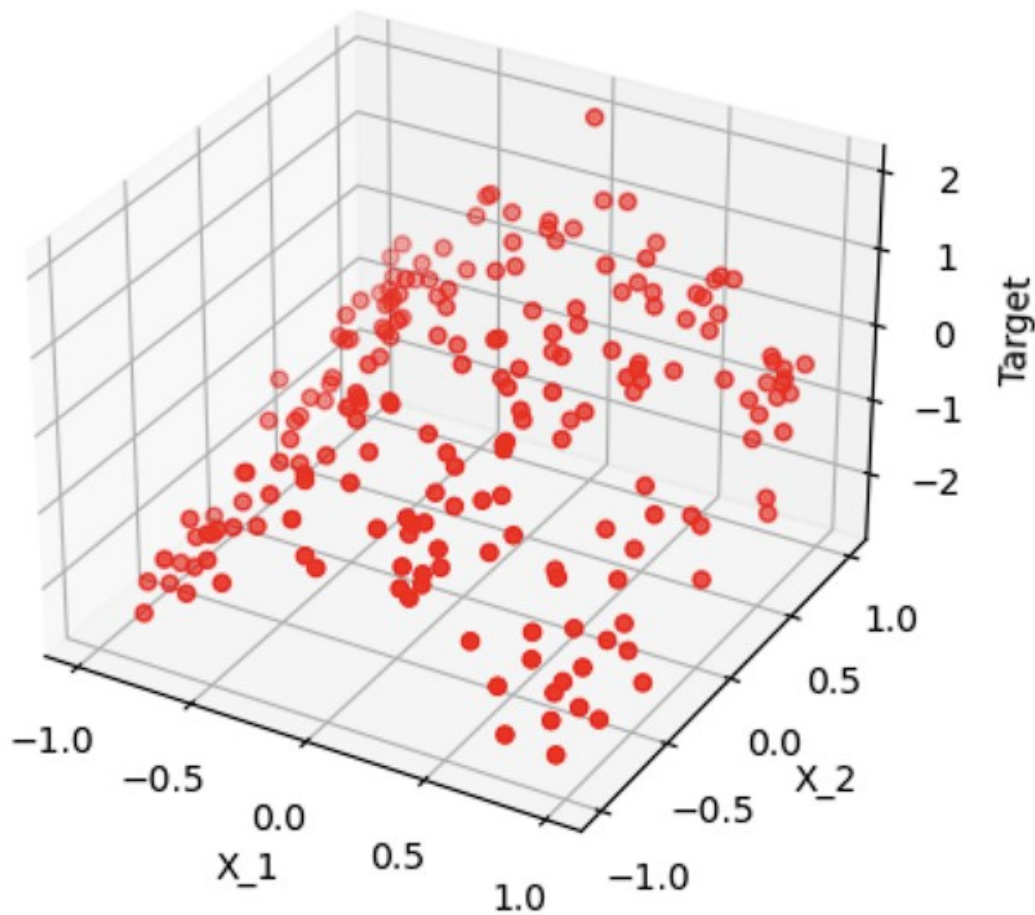
Code Index: 2

First thing we do is to read and load the csv file into a variable using the pandas library using `pd.read_csv`. Afterwards, we load the first two columns of the csv file, which are the X feature values into a variable and similarly, we load the target features column Y, which is the last column (3rd column or 2 by index) into the y variable.

Now using the matplotlib library, first I specified I called the `.figure()` to initialize and then defined the the plot to be 3d using the projection = '3d' specification in the matplotlib library. Similar to the previous assignment, I defined the 3D plot to be a scatterplot and passed through the X and Y features and set color of the points to be red and then the marker types as o, to make spherical points on the 3D plots.

Afterwards I set the features of the plot by naming the axes of the plot using `.label` command and `.show` to visualize the 3D plot.

The Output



Here in this 3D Plot, The X axis of the graph specifies the X_1 feature values, the Y-Axis specifies the X_2 feature values and the Z-axis(vertical axis) specifies the target values Y.

The points visualized in this 3D space take on the axis values of all 3 axes to plot the point on the graph, from looking at the graph, it (for now) seems to have clusters between a few ranges, particularly between X_1 = 0.5 and X_2 = -0.5 and looks to be non- linear. However, further steps would be needed to confirm this and see the work of the values.

Part(i).b

Code Index = 3

At first I defined the array of C values that would be used for the model, as specified by the assignment, I used the values C values =1,10,1000. The lower the values of C value is, the higher the regularization is (Regularization being the penalty (like cost functions) added to the model). Next I defined the Model, being PolynomialFeatures with the degree 5, which means the polynomial features generated by the mode (X_1⁵, X_1⁴...).

Then I transformed the features into the polynomial feature space, so the new matrix contains the new polynomial degrees needed for the model. Then to make the lasso model, inside the for loop of C_values, I first specified the alpha value ($1/(2*C)$) that is used in the Lasso model as a regularization parameter (as

discussed before, it is inverse of C as seen in the function before), and then passed it into the LassoModel with iterations of 10000, so the model has time to converge.

After this, I passed the feature parameters into the model using the .fit function. Then to get the values of the model, I used the .coef_ function to get the parameter values. Afterwards, to get the names of the features while printing, I used feature_name variables and the .get_feature_names_out to get their names while printing the values. And to print the the models, I used another nested for loop to print the feature and the coefficient respectively.

The Output

```
Model for C=1:
1,: 0.0
x1,: 0.0
x2,: 0.0
x1^2,: -0.0
x1 x2,: 0.0
x2^2,: 0.0
x1^3,: -0.0
x1^2 x2,: 0.0
x1 x2^2,: -0.0
x2^3,: 0.0
x1^4,: -0.0
x1^3 x2,: 0.0
x1^2 x2^2,: -0.0
x1 x2^3,: 0.0
x2^4,: 0.0
x1^5,: -0.0
x1^4 x2,: 0.0
x1^3 x2^2,: -0.0
x1^2 x2^3,: 0.0
x1 x2^4,: -0.0
x2^5,: 0.0
Model for C=10:
1,: 0.0
x1,: 0.0
x2,: 0.8493203514770375
x1^2,: -1.4612305047530285
x1 x2,: 0.0
x2^2,: 0.0
x1^3,: 0.0
x1^2 x2,: 0.0
x1 x2^2,: 0.0
x2^3,: 0.0
x1^4,: -0.0
x1^3 x2,: -0.0
x1^2 x2^2,: -0.0
x1 x2^3,: 0.0
x2^4,: 0.0
x1^5,: -0.0
x1^4 x2.: 0.0
```

```

x1^3 x2^2,: -0.0
x1^2 x2^3,: 0.0
x1 x2^4,: -0.0
x2^5,: 0.0
Model for C=1000:
1,: 0.0
x1,: 0.040182104452646104
x2,: 0.8906599185448412
x1^2,: -2.099847832142753
x1 x2,: -0.0
x2^2,: 0.0
x1^3,: 0.0
x1^2 x2,: 0.0
x1 x2^2,: -0.0
x2^3,: -0.0
x1^4,: 0.0902988470119717
x1^3 x2,: -0.026113820547932605
x1^2 x2^2,: -0.0
x1 x2^3,: 0.02054777631720162
x2^4,: 0.04744551839196259
x1^5,: -0.0
x1^4 x2,: 0.22830864957327615
x1^3 x2^2,: -0.0
x1^2 x2^3,: -0.31876334744166096
x1 x2^4,: -0.030834107316712298
x2^5,: 0.25545585972378293

```

Now from the output, X_1 and X_2 are the original features. X_1^2, X_2^3 ...etc are the higher powers of the X_1 and X_2 features and the coefficients are $X_1^2 * X_2^3$ are the interaction features between X_1 and X_2 .

Now when we see the values of C , as explained before, the lower the C , the higher the regularization. When we see $C = 1$, it has high regularization, by seeing the values, they are all almost zero or zero. This tells us that the regularization has suppressed the features to almost zero,. Similarly, for $C = 10$, it has moderate levels of regularization, as we see the original feature values have moderate values of the model, however the model still makes the complex features become close to zero. And now similarly, for $C=1000$, we see low levels of regularization, now the complex terms have non-zero values telling us that the model is now able to use the more complex features in the model.

Part (i). (c)

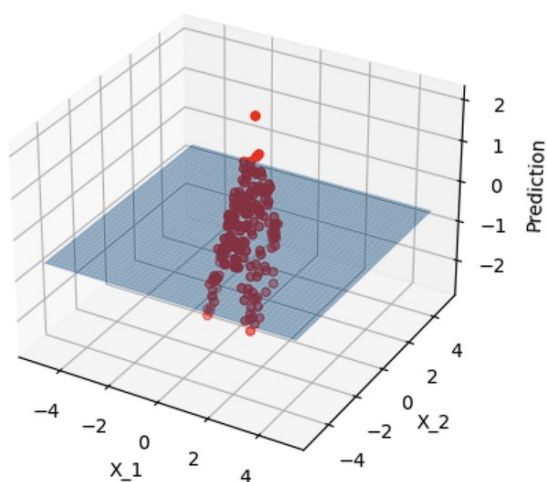
[Code Index = 4](#)

As specified in the assignment, I used `grid = np.linspace` that would create a grid that goes from values -5 to 5 to visualize the predictions. Now to transform the test data to be used in the model similar to how it was previously done to use polynomial feature space.

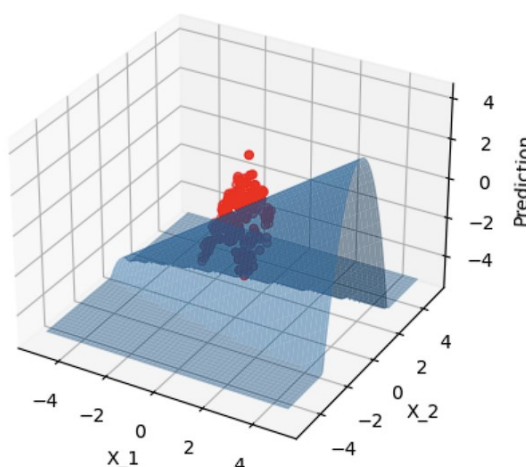
As done before, using `.figure`, `.add_subplot` and `.scatter` to initialize and specify the 3D scatter plot model. Now using the same array of `C_values`, `alpha` and fitting them in the Lasso model as done in the last part, we use `y_pred` to make predictions from the model, using `.predict` and using `.reshape` to match the grid. I also used `np.clip` to clip the predictions to a range of -5 to 5 as the plot was being distorted and uneven due to some extreme outliers as before. Now as done in the last step, I used `.figure`, `.add_subplot` and `.scatter` to visualize the plot and the markers. I also used `.plot_surface` and `np.meshgrid` to make the array grid surface and set the transparency to 0.5 (alpha value), to be able to see underlying data points.

The Output

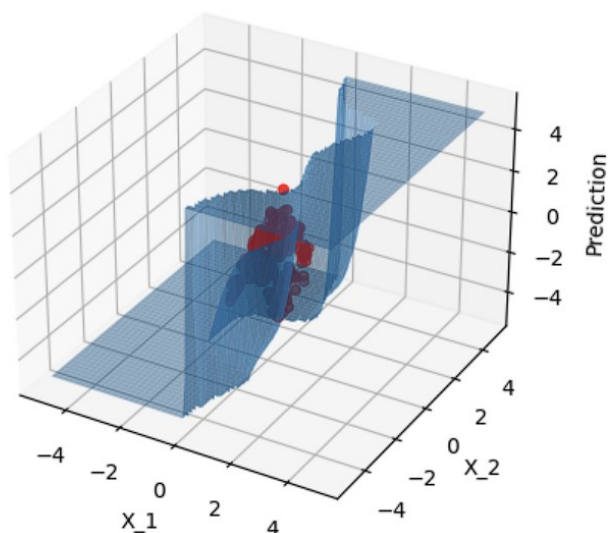
Lasso Predictions for C = 1



Lasso Predictions for C = 10



Lasso Predictions for C = 1000



Now as we observe the plots, we can see the effect that the regularization and C values have on the predictions. For C=1, High regularization, we can see that the prediction surface is relatively flat as the high regularization is suppressing the features of the model.

For C=10, Moderate Regularization, The model prediction surface has a curved shape now, which better fits the data points of the model, as well as capturing more variance in the data.

Now for $C = 1000$, Low Regularization, the model prediction surface is more jagged and captures complex features even better, trying to fit every data point in the model.

In the next part, I will discuss more on what this prediction surface indicates.

Part(i).D

Underfitting is when the model is too simple to capture the complex relationships in the data and fails to fit the training data while overfitting is when the model becomes too complex and starts fitting the noise/outliers in the data and kind of 'memorizes' the data and performs poorly on new data for predictions.

Now leading from the last part, as seen in the plots, when there is a low Value of C ($C = 1$), hence high regularization, the model is underfitting the data, as we see, the prediction mesh does not encapsulate the data given and underfits the model. While we observe overfitting in the high value of C model ($C=1000$). Where we see the model tries too hard to fit the training data, as seen by the jagged lines in the prediction mesh.

Hence we need to have the correct, or close to correct value of C in our model to avoid the model from underfitting or overfitting the data, the closest we see this in the previous part is when $C=10$, where it captures the data quite well comparatively and we do not observe any underfitting or overfitting of the data.

Part(i).E

Code Index: 5

The code is very similar, almost identical to the Lasso Model, however this time we are using the Ridge Prediction Model. The Lasso model uses L1 penalty, however Ridge uses L2 penalty which does not drive the squared coefficients to drive like L1 penalty does.

L1 penalty adds penalty to the absolute value of coefficients while L2 adds a penalty to the squared value of the coefficients. So as observed, L1 can make some coefficient values be zero, which effectively removes them.

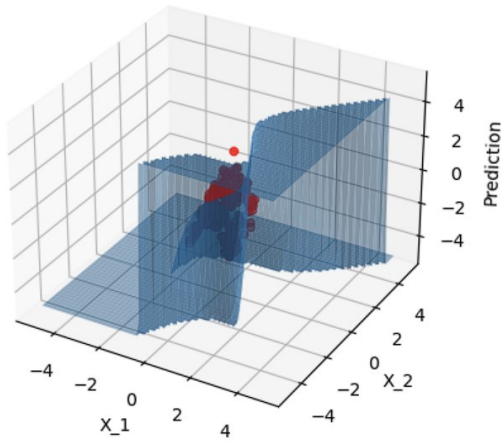
Now onto the code, the code is near identical to the last part, the difference is in calling the model and its parameters.

```
ridge_model = Ridge(alpha=alpha, max_iter=20000, tol=0.001)
```

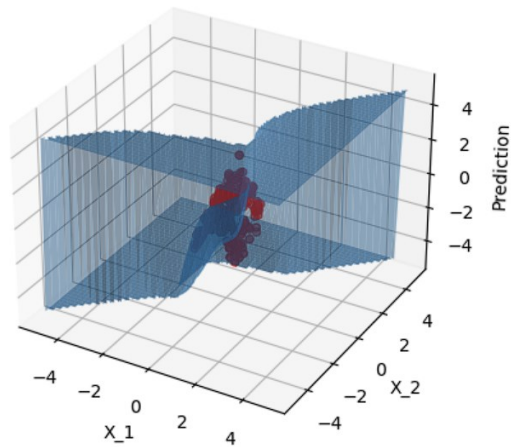
this line calls the ridge model, passing the alpha value with 20000 iterations (needed increase to converge the value), the tolerance passed is 0.001, which is basically the minimum change in the cost function per iteration to converge the values. The smaller the tolerance value, the more precise the model is but takes more iterations to converge while a larger tolerance value would sacrifice precision over taking less iterations.

The Output

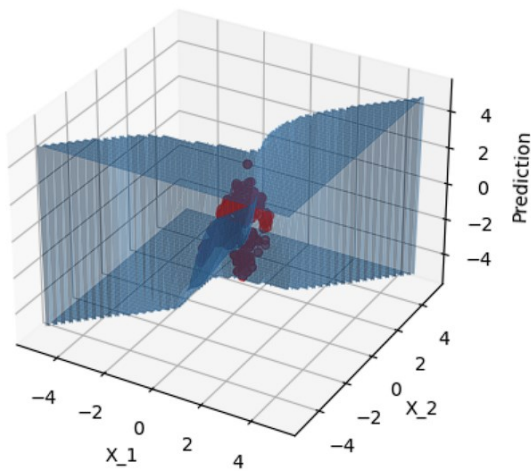
Ridge Predictions for C = 1



Ridge Predictions for C = 10



Ridge Predictions for C = 1000



Ridge for C=1:

```
1: 0.0
x1: 0.046939803728794575
x2: 0.843254677695572
x1^2: -1.6447572299480977
x1 x2: -0.01140793625285304
x2^2: 0.041691984412710874
x1^3: 0.08782487383119471
x1^2 x2: 0.02290824363197302
x1 x2^2: -0.01832661902267905
x2^3: 0.07734977455975024
x1^4: -0.30022192544724785
x1^3 x2: -0.02038175114149919
x1^2 x2^2: -0.21794567018922878
x1 x2^3: 0.042449801357195394
x2^4: 0.07683160321633269
x1^5: -0.10702359313592587
x1^4 x2: 0.22286272753714562
x1^3 x2^2: 0.07886321540357435
x1^2 x2^3: -0.2863783349183977
x1 x2^4: -0.09421302257263092
x2^5: 0.20988567228721017
```

Ridge for C=10:

```
1: 0.0
x1: -0.031681289306698994
x2: 0.9405966823948853
x1^2: -2.1502613799514214
x1 x2: -0.02775156827665331
x2^2: 0.0656844227591196
x1^3: 0.4357175347232298
x1^2 x2: -0.016737795394374796
x1 x2^2: -0.06319565677547138
x2^3: -0.2724734363209529
x1^4: 0.16934442799363303
x1^3 x2: -0.09677271369008973
x1^2 x2^2: -0.03358731530779903
x1 x2^3: 0.1295820039238629
x2^4: -0.004423076208629541
```

```

x1^5: -0.4069449587589172
x1^4 x2: 0.45150024731241845
x1^3 x2^2: 0.029947350898528756
x1^2 x2^3: -0.5536602979205241
x1 x2^4: -0.03540405016944812
x2^5: 0.546832058956932

```

Ridge for C=1000:

```

1: 0.0
x1: -0.08738941361736373
x2: 0.9860865016578773
x1^2: -2.266348941360275
x1 x2: -0.02503305548218904
x2^2: 0.09552495126365454
x1^3: 0.6851737182740355
x1^2 x2: -0.01943084909890659
x1 x2^2: -0.08658105383707067
x2^3: -0.46822558026311506
x1^4: 0.2798375812449822
x1^3 x2: -0.12224259919432387
x1^2 x2^2: 0.012355973117365639
x1 x2^3: 0.14573239099146865
x2^4: -0.054387159444890004
x1^5: -0.6117652487756823
x1^4 x2: 0.5215041571004427
x1^3 x2^2: -0.023210353648349735
x1^2 x2^3: -0.6343676478191748
x1 x2^4: 0.014154824248831719
x2^5: 0.7288165560630137

```

Here we see the Ridge predictions for the model, similar to the Lasso Predictions, lower regularization(High C values) make for better fitting of the training data at the risk of overfitting while the inverse is true for High Regularization(Low C values).

Since Ridge prediction does not eliminate some features by equating them to zero and instead reduces their impact, we see better fitting of data and more jagged lines due to complex polynomials in effect.

In Comparison to the Lasso Model, we see that the Ridge Model fits the data better in comparison even for the lower values of C.

Part ii. (a)

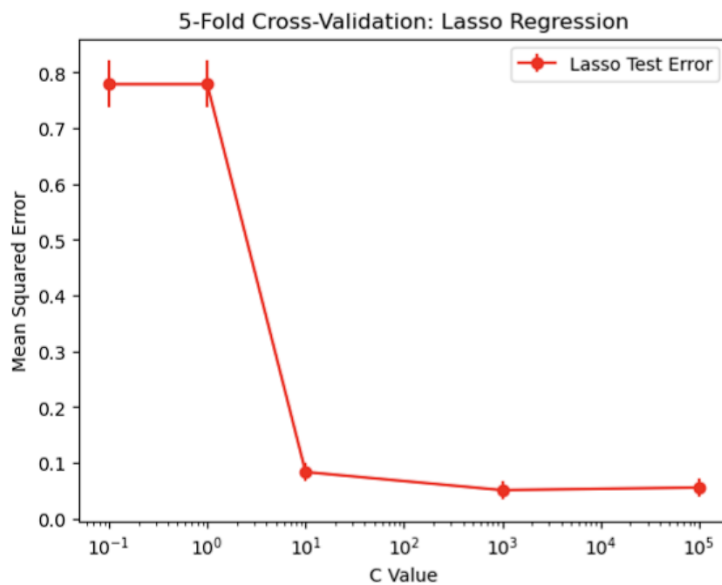
[Code Index: 6](#)

In this code, first I defined the array of C values and initialized the array of mean error and standard error values as well. Then I set up the K-Fold Cross-Validation, by initializing the kfold function and specifying 5 splits, which means to do a 5-fold cross validation, which basically divides the data into 5 equal parts, with in each iteration, 4 of the parts are used for training and the other 1 part is used for testing till all parts have been tested once, giving us 5 data points to plot.

Now to use the cross validation, I set up a function for cross validation with the model, feature and target values and put in a loop for the cross validation sets to be tested using the feature data and target data loaded and then using model.fit to fit the data and then model.predict to make predictions based on the test data, then we get the mean_squared_error by comparing the true target values against the model's prediction and then append the MSE values into the mse array initialized earlier.

After this we plot errorbar to show the MSE values using matplotlib and similar to earlier we used matplotlib library to visualize the graph.

The Output



In this plot, we see the MSE plotted against the C-values in the graph, as we can see on the graph, at first the low value of the C leads to a high MSE, now from 1 to 10 values of C, the C value sharply decreases stabilizing at around 1000 and then slightly increasing at 100000.

The high value of MSE at low C might indicate underfitting of the model and after $C=10$ it seems to stabilize at this point with minimal changes.

Part (ii). (b)

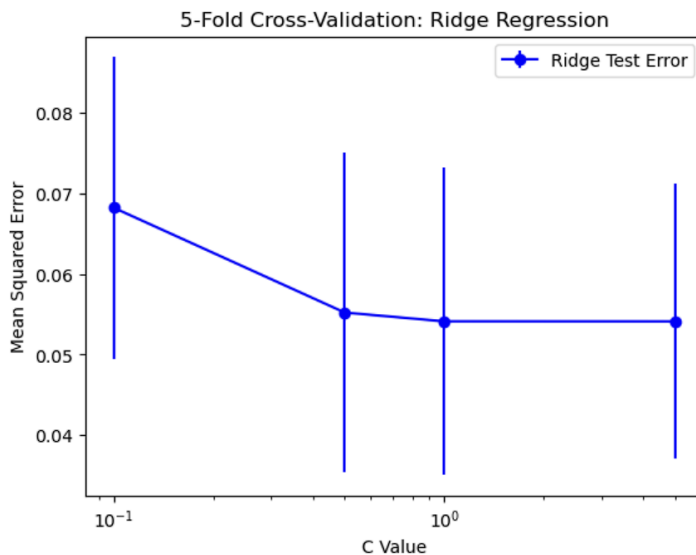
I think the optimal value of C for this dataset and model would lie around $C=10$ to $C=1000$, from the graph, I think the most optimal value of would be 100, however C in the aforementioned range would suffice due to as seen in graph, the values of C before $C=10$ sharply decrease and after $C=10$, there is minimal change in the MSE values. Although $C=1000$ seems to be the lowest point in the graph, I believe $C=100$ would at least not take on much risk of overfitting as other high values of C while also leveraging the slight decrease in MSE between $C=10$ to $C=1000$.

Part(ii).(C)

Code Index: 7

This code is identical to the one discussed in part ii.a except the ridge regression model is used in place of the lasso model used before.

The Output



In this plot, we see the MSE plotted against the C-values in the graph for the Ridge regression model. As observed, initially, the low value of C leads to a relatively high MSE, suggesting underfitting. From $C = 0.1$ to $C = 1$, the MSE gradually decreases, indicating that as regularization weakens, the model's performance improves. After $C = 1$, the MSE stabilizes with minimal changes across higher C values, showing that the model has reached a point where further increasing C does not decrease the MSE much.

The high MSE at low C values likely indicates underfitting due to the stronger regularization, which restricts the model from capturing the data properly. The MSE stabilizes around $C = 1$, showing that the model achieves a balance between underfitting and overfitting. While there is little difference in MSE between C values from 1 to 10, I think the best value of C for this dataset and model lies around $C = 1$. This value avoids overfitting while providing a minimal MSE as in my observation of the lasso model graph. However, C values between 1 and 10 would also suffice as the MSE stabilizes beyond $C = 1$ with little change in performance.

CODE INDEX

1.

```
import pandas as pd
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import numpy as np
from sklearn.linear_model import Lasso, Ridge
from sklearn.model_selection import KFold
from sklearn.metrics import mean_squared_error
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
```

2.

```
df = pd.read_csv('dataset3_1.csv', header=None)

X = df[[0, 1]].values
y = df[2].values

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

ax.scatter(X[:, 0], X[:, 1], y, c='red', marker='o')

ax.set_xlabel('X_1')
ax.set_ylabel('X_2')
ax.set_zlabel('Target')

plt.show()
```

3.

```
C_values = [1, 10, 1000]

poly = PolynomialFeatures(degree=5)
X_poly = poly.fit_transform(X)
```

```

for C in C_values:
    alpha = 1 / (2 * C)
    lasso_model = Lasso(alpha=alpha, max_iter=10000)
    lasso_model.fit(X_poly, y)
    coefs = lasso_model.coef_

    feature_names = poly.get_feature_names_out(['x1', 'x2'])

    print(f"Model for C={C}:")
    for feature, coef in zip(feature_names, coefs):
        print(f"{feature},: {coef}")

```

4.

```

grid = np.linspace(-5, 5, 100)
Xtest = np.array([[i, j] for i in grid for j in grid])
Xtest_poly = poly.transform(Xtest)
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.scatter(X[:, 0], X[:, 1], y, color='red', label='Training Data')

ax.set_xlabel('X_1')
ax.set_ylabel('X_2')
ax.set_zlabel('Prediction')
plt.title('Training Data in 3D')
plt.show()

C_values = [1, 10, 1000]
for C in C_values:
    alpha = 1 / (2 * C)

    lasso_model = Lasso(alpha=alpha, max_iter=20000, tol=0.001)
    lasso_model.fit(X_poly, y)

    y_pred = lasso_model.predict(Xtest_poly)

```

```

y_pred = y_pred.reshape((100, 100))
y_pred_clipped = np.clip(y_pred, a_min=-5, a_max=5)

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.scatter(X[:, 0], X[:, 1], y, color='red', label='Training Data')
ax.plot_surface(np.meshgrid(grid, grid)[0], np.meshgrid(grid, grid)[1],
y_pred_clipped, alpha=0.5)

ax.set_xlabel('X_1')
ax.set_ylabel('X_2')
ax.set_zlabel('Prediction')
plt.title(f'Lasso Predictions for C = {C}')
plt.show()

```

5.

```

grid = np.linspace(-5, 5, 100)
Xtest = np.array([[i, j] for i in grid for j in grid])

Xtest_poly = poly.transform(Xtest)

C_values = [1, 10, 1000]

for C in C_values:
    alpha = 1 / (2 * C)

    ridge_model = Ridge(alpha=alpha, max_iter=20000, tol=0.001)

    ridge_model.fit(X_poly, y)

    y_pred = ridge_model.predict(Xtest_poly)

    y_pred = y_pred.reshape((100, 100))

    y_pred_clipped = np.clip(y_pred, a_min=-5, a_max=5)

```

```

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.scatter(X[:, 0], X[:, 1], y, color='red', label='Training Data')
ax.plot_surface(np.meshgrid(grid, grid)[0], np.meshgrid(grid, grid)[1],
y_pred_clipped, alpha=0.5)

ax.set_xlabel('X_1')
ax.set_ylabel('X_2')
ax.set_zlabel('Prediction')
plt.title(f'Ridge Predictions for C = {C}')
plt.show()

for C in C_values:
    alpha = 1 / (2 * C)

    ridge_model = Ridge(alpha=alpha, max_iter=20000, tol=0.001)
    ridge_model.fit(X_poly, y)

    coefs = ridge_model.coef_

    feature_names = poly.get_feature_names_out(['x1', 'x2'])

    print(f"Ridge for C={C}:")
    for feature, coef in zip(feature_names, coefs):
        print(f"{feature}: {coef}")
    print("\n")

```

6.

```
C_range = [0.1, 10, 10, 1000, 10000]
```

```
mean_error_lasso = []
```

```
std_error_lasso = []
```

```
kf = KFold(n_splits=5)
```

```

def cross_val_model_lasso(model, X_poly, y):
    temp_train_errors = []
    temp_test_errors = []

    for train_idx, test_idx in kf.split(X_poly):
        X_train, X_test = X_poly[train_idx], X_poly[test_idx]
        y_train, y_test = y[train_idx], y[test_idx]

        model.fit(X_train, y_train)

        y_train_pred = model.predict(X_train)
        y_test_pred = model.predict(X_test)

        train_mse = mean_squared_error(y_train, y_train_pred)
        test_mse = mean_squared_error(y_test, y_test_pred)

        temp_train_errors.append(train_mse)
        temp_test_errors.append(test_mse)

    return np.mean(temp_train_errors), np.std(temp_train_errors),
    np.mean(temp_test_errors), np.std(temp_test_errors)

for C in C_range:
    alpha = 1 / (2 * C)

    lasso_model = Lasso(alpha=alpha, max_iter=10000)
    mean_train, std_train, mean_test, std_test =
cross_val_model_lasso(lasso_model, X_poly, y)

    mean_error_lasso.append(mean_test)
    std_error_lasso.append(std_test)

plt.errorbar(C_range, mean_error_lasso, yerr=std_error_lasso, label='Lasso Test
Error', fmt='-o', color='red')

plt.xscale('log')

```

```
plt.xlabel('C Value')
plt.ylabel('Mean Squared Error')
plt.title('5-Fold Cross-Validation: Lasso Regression')
plt.legend()
plt.show()
```

7.

```
C_range = [0.1, 1, 10, 1000, 100000]
```

```
mean_error_ridge = []
std_error_ridge = []
```

```
kf = KFold(n_splits=5)
```

```
def cross_val_model_ridge(model, X_poly, y):
    temp_train_errors = []
    temp_test_errors = []

    for train_idx, test_idx in kf.split(X_poly):
        X_train, X_test = X_poly[train_idx], X_poly[test_idx]
        y_train, y_test = y[train_idx], y[test_idx]

        model.fit(X_train, y_train)

        y_train_pred = model.predict(X_train)
        y_test_pred = model.predict(X_test)

        train_mse = mean_squared_error(y_train, y_train_pred)
        test_mse = mean_squared_error(y_test, y_test_pred)

        temp_train_errors.append(train_mse)
        temp_test_errors.append(test_mse)

    return np.mean(temp_train_errors), np.std(temp_train_errors),
    np.mean(temp_test_errors), np.std(temp_test_errors)
```

```
for C in C_range:
    alpha = 1 / (2 * C)

    ridge_model = Ridge(alpha=alpha, max_iter=10000)
    mean_train, std_train, mean_test, std_test =
cross_val_model_ridge(ridge_model, X_poly, y)

    mean_error_ridge.append(mean_test)
    std_error_ridge.append(std_test)

plt.errorbar(C_range, mean_error_ridge, yerr=std_error_ridge, label='Ridge Test
Error', fmt='-o', color='blue')
plt.xscale('log')
plt.xlabel('C Value')
plt.ylabel('Mean Squared Error')
plt.title('5-Fold Cross-Validation: Ridge Regression')
plt.legend()
plt.show()
```